

Task 2 — E-Commerce Product Search & Checkout

Frontend Engineering • Next.js App Router / TypeScript / React / Tailwind CSS

Build a high-performance e-commerce application with product browsing, cart, and checkout.

Requirements

- Product listing with 500+ products.
- Search, category, price, rating, sorting, and pagination.
- URL-based filters with refresh persistence.
- Product details with images, stock, reviews, and related products.
- Shopping cart with add, remove, quantity management, and persistence.
- Checkout form using React Hook Form and Zod.
- SEO-friendly product pages.
- Loading, empty, and error states.

Data & API

- Use a JSON dataset or mock API to provide 500+ products.
- Create a proper API service layer for product listing, product details, and related products.
- Use query parameters for search, filters, sorting, and pagination.
- Keep API logic separate from components.
- Handle loading, errors, empty results, and invalid product requests.
- Avoid fetching unnecessary data and duplicate API calls.

Architecture & React

- Use Next.js App Router with appropriate Server and Client Components.
- Use URL search parameters for product filters.
- Use Zustand or another suitable state management solution for the cart.
- Use `useEffect` only for required side effects, such as persistence or client synchronization.
- Use `useMemo` for expensive derived calculations, such as cart totals or filtered data, when justified.
- Use `useCallback` and `React.memo` where they provide meaningful render optimization.
- Avoid unnecessary state, unnecessary re-renders, and excessive client-side JavaScript.
- Follow proper React lifecycle and cleanup practices.

Evaluation

- Next.js rendering strategy.
- API architecture and data fetching.
- State management and URL state.
- React performance and lifecycle understanding.
- Component design and reusability.
- SEO and responsive UI/UX.
- Clean architecture and design patterns.

- AI-assisted development quality.

Submission Requirements

- 1 GitHub repository.
- 2 README with setup instructions.
- 3 Architecture and folder structure explanation.
- 4 API/data-fetching approach.
- 5 Explanation of Server vs Client Components.
- 6 Performance decisions: useMemo, useCallback, memoization, and rendering optimization.
- 7 Working demo if available.

Note: Candidates may use AI tools, but they should understand the generated code and be able to explain their architectural and technical decisions.